Data Preprocessing

Creating Training and Test Sets in Machine Learning

Splitting a dataset into **training** and **test** sets is a fundamental step in machine learning. It ensures that a model is trained on one subset and evaluated on another to measure generalization performance.

1. Why Split Data?

- **Training Set** → Used to train the model.
- **Test Set** → Used to evaluate the model's performance on unseen data.
- Helps **prevent overfitting** by ensuring the model is tested on new data.

2. Standard Train-Test Split Using Scikit-Learn

The common split ratio is **80% for training** and **20% for testing**.

```
from sklearn.model_selection import train_test_split
import numpy as np

# Sample dataset
X = np.arange(10).reshape((5, 2))  # Feature matrix
y = np.array([0, 1, 0, 1, 0])      # Target labels

# Splitting dataset (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print("Training Features:\n", X_train)
print("Testing Features:\n", X_test)
```

✅ **random_state=42** ensures reproducibility.

3. Stratified Sampling for Imbalanced Data

If the dataset is **imbalanced**, **stratified sampling** ensures class distribution remains the same in both sets.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)
```

✔ **Stratified Sampling** maintains class proportions across train-test sets.

4. Train-Validation-Test Split

For better performance evaluation, datasets are often split into **three parts**:

- **Training Set** (70%) → Used for training.
- **Validation Set** (15%) → Used for hyperparameter tuning.
- **Test Set** (15%) → Used for final evaluation.

```
X_train, X_temp, y_train, y_temp = train_test_split(X, y, test_size=0.3, random_state=42)
X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, test_size=0.5, random_state=42)
```

5. Cross-Validation (CV) Instead of a Single Split

Instead of one train-test split, **cross-validation (CV)** improves robustness by training multiple times on different splits.

```
from sklearn.model_selection import KFold, StratifiedKFold

kf = KFold(n_splits=5, shuffle=True, random_state=42)  # 5-Fold CV
strat_kf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

for train_index, test_index in kf.split(X):
    print("Train:", train_index, "Test:", test_index)
```

✔ **Cross-validation** provides a more reliable performance estimate.

Conclusion

- **Basic Split**: train_test_split()
- **Stratified Split** for **imbalanced data**.
- **Train-Validation-Test Split** for hyperparameter tuning.
- **Cross-Validation (CV)** for robust evaluation.

Feature Scaling

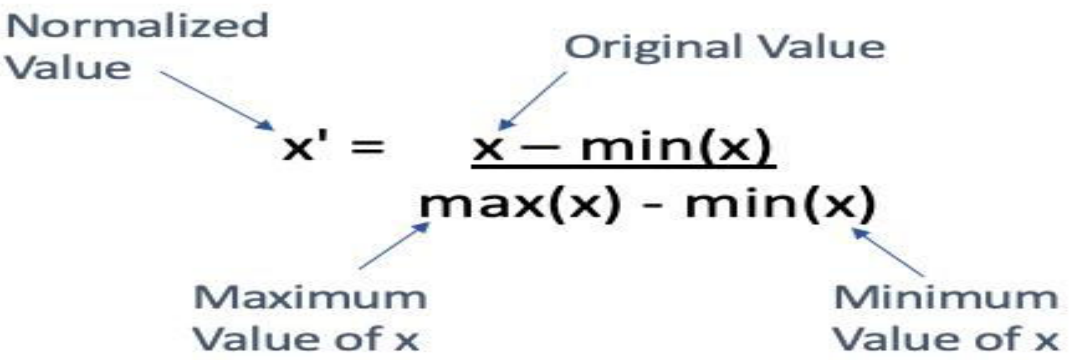
Feature scaling is the process of normalizing or standardizing the range of numerical features so that they contribute equally to the model. This is especially important for algorithms that rely on distance calculations (e.g., KNN, SVM) or gradient-based optimization (e.g., neural networks).

Why Feature Scaling?

- Ensures that features with larger scales do not dominate those with smaller scales.
- Improves convergence speed in gradient-based algorithms.
- Enhances model performance and stability.

Types of Feature Scaling:

1. **Normalization (Min-Max Scaling):**
    - Scales features to a fixed range, usually [0, 1].
    - **Formula:**



- **Example:** Scaling pixel values (0-255) in images to [0, 1].

2. **Standardization (Z-Score Scaling):**
   - Scales features to have a mean of 0 and a standard deviation of 1.
   - **Formula:**

$$X_i' = \frac{X_i - \mu}{\sigma} = \frac{X_i - X_{\text{mean}}}{X_{std}}$$

StandardScaler

- **Example:** Scaling features like age, income, or height for regression models.

3. **Robust Scaling:**
   - Uses the median and interquartile range (IQR) to scale features, making it robust to outliers.
   - **Formula:**

Robust Standardised Value

Original Value

Sample Median

$$x' = \frac{x - \text{median}(x)}{(Q3-Q1)}$$

Interquartile Range = Q3 – Q1

- **Example:** Scaling data with outliers, such as income data.

Feature Engineering: Enhancing Machine Learning Models

Feature engineering is the process of **creating, transforming, and selecting features** to improve the performance of a machine learning model.

1. Why is Feature Engineering Important?

- Improves model **accuracy** and **performance**.
- Reduces **overfitting** by removing irrelevant features.
- Helps models **learn better patterns** from data.
- Reduces **training time** by eliminating unnecessary features.

2. Types of Feature Engineering

(A) Creating New Features

Adding new **derived** features to improve learning.

🔹 **Example: Extracting Date Features**

```
import pandas as pd

df = pd.DataFrame({'date': ['2023-01-01', '2023-06-15', '2023-12-31']})
df['date'] = pd.to_datetime(df['date'])  # Convert to datetime
df['year'] = df['date'].dt.year
df['month'] = df['date'].dt.month
df['day'] = df['date'].dt.day
df['weekday'] = df['date'].dt.weekday

print(df)
```

✔ Helps models learn seasonal trends.

🔹 **Example: Feature Interaction (Multiplying Features)**

```
df['area'] = df['length'] * df['width']  # Creating an 'area' feature
```

✔ Captures **relationships** between features.

(B) Modifying Features

Changing existing features to improve model performance.

◆ **1. Log Transformation (Handling Skewed Data)**

```
import numpy as np
df['log_price'] = np.log1p(df['price'])  # log(1 + price)
```

✔ Reduces the effect of **outliers** in price data.

✔ Reduces the effect of **outliers** in price data.

◆ **2. Scaling Features (Standardization & Normalization)**

Standardization (Z-score Normalization):

$$X_{scaled} = \frac{X - \text{mean}(X)}{\text{std}(X)}$$

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
df['scaled_price'] = scaler.fit_transform(df[['price']])
```

✔ Useful for **linear models, SVMs, and PCA**.

- **Min-Max Scaling (0 to 1 Range):**

```
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()
df['normalized_price'] = scaler.fit_transform(df[['price']])
```

- ✔ Useful for **Neural Networks & Gradient Descent models**.
-

(C) Handling Missing Data

- 🔹 **1. Filling with Mean/Median/Mode**

```
df['age'].fillna(df['age'].mean(), inplace=True)  # Replace missing with mean
```

🔹 **2. Using Forward/Backward Fill (for Time Series)**

```
df['temperature'].fillna(method='ffill', inplace=True)  # Forward fill
```

(D) Encoding Categorical Variables

🔹 **1. One-Hot Encoding**

```
from sklearn.preprocessing import OneHotEncoder

encoder = OneHotEncoder(sparse_output=False)
encoded = encoder.fit_transform(df[['category']])
```

✔ Best for **Nominal** (unordered) categories.

🔹 **2. Label Encoding**

```
from sklearn.preprocessing import LabelEncoder

encoder = LabelEncoder()
df['category_encoded'] = encoder.fit_transform(df['category'])
```

✔ Best for **Ordinal** (ordered) categories.

3. Feature Selection (Removing Unnecessary Features)

🔹 **Removing Low-Variance Features**

```
from sklearn.feature_selection import VarianceThreshold

selector = VarianceThreshold(threshold=0.1)  # Removes features with variance < 0.1
df_selected = selector.fit_transform(df)
```

✔ Helps remove **redundant** features.

🔹 **Using Feature Importance (Random Forest)**

```
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier()
model.fit(X_train, y_train)
importances = model.feature_importances_
```

✔ Helps identify **important** features.

Conclusion

✅ Feature Engineering is **crucial** for improving model performance.
✅ Includes **creating new features, modifying data, and selecting important features**.
✅ Helps **reduce training time** and **improve accuracy**.

Data Cleaning – Treating Missing Values and Outliers

Data cleaning is a critical step in the data preprocessing pipeline. It involves identifying and addressing issues like **missing values** and **outliers** to ensure the data is accurate, consistent, and ready for analysis or modeling. Poor data quality can lead to incorrect conclusions and poor model performance.

Treating Missing Values

Missing values occur when no data is stored for certain features in a dataset. They can arise due to errors in data collection, processing, or storage. Handling missing values is essential because most machine learning algorithms cannot work with incomplete data.

Common Techniques to Handle Missing Values:

1. **Remove Missing Data**:
   - Drop rows or columns with missing values.
   - Use when the amount of missing data is small and random.

Example:

```python
df.dropna()  # Drop rows with missing values
df.dropna(axis=1)  # Drop columns with missing values
```

2. **Imputation**:
   - Fill missing values with estimated or calculated values.
   - Common methods:
     - **Mean/Median/Mode Imputation**: Replace missing values with the mean (for numerical data), median (for skewed data), or mode (for categorical data).

```python
df['column'].fillna(df['column'].mean(), inplace=True)
```

- **Forward/Backward Fill:** Use the previous or next value in the dataset (common in time-series data).

```python
df['column'].fillna(method='ffill')  # Forward fill
df['column'].fillna(method='bfill')  # Backward fill
```

- **Predictive Imputation:** Use machine learning models (e.g., regression, KNN) to predict missing values.
-

3. **Flagging Missing Values:**
   - Create a new binary feature to indicate whether a value was missing.

```python
df['column_missing'] = df['column'].isnull().astype(int)
df['column'].fillna(df['column'].mean(), inplace=True)
```

4. **Advanced Techniques:**
   - Use algorithms like MICE (Multiple Imputation by Chained Equations) for complex datasets.
   - Use deep learning models for imputation in high-dimensional data.

Treating Outliers

Outliers are data points that deviate significantly from the rest of the data. They can arise due to errors (e.g., data entry mistakes) or natural variability (e.g., extreme values in a distribution). Outliers can skew analysis and negatively impact model performance.

Common Techniques to Handle Outliers:

1. **Visual Inspection:**
   - Use plots like boxplots, scatterplots, or histograms to identify outliers.

```python
import seaborn as sns
sns.boxplot(df['column'])
```

Statistical Methods:

- Z-Score: Identify outliers as data points with a Z-score greater than a threshold (e.g., |Z| > 3).

```python
from scipy.stats import zscore
df['z_score'] = zscore(df['column'])
outliers = df[abs(df['z_score']) > 3]
```

IQR (Interquartile Range): Identify outliers as data points outside 1.5 * IQR from the quartiles.

```python
Q1 = df['column'].quantile(0.25)
Q3 = df['column'].quantile(0.75)
IQR = Q3 - Q1
outliers = df[(df['column'] < (Q1 - 1.5 * IQR)) | (df['column'] > (Q3 + 1.5 * IQR))]
```

Removing Outliers:

- **Drop rows containing outliers.**

```python
df = df[(df['column'] >= lower_bound) & (df['column'] <= upper_bound)]
```

Capping/Truncating:

- Replace outliers with a specified upper or lower limit.

```python
df['column'] = df['column'].clip(lower=lower_bound, upper=upper_bound)
```

Transforming Data:

- Apply transformations like log, square root, or Box-Cox to reduce the impact of outliers.

```python
df['column'] = np.log(df['column'])
```

Binning:

- Convert continuous data into discrete bins to reduce the effect of outliers.

```python
df['column_bin'] = pd.cut(df['column'], bins=[0, 10, 20, 30, 40, 50])
```

Scenario: Cleaning a Sales Dataset

Consider a dataset containing sales data with missing values and outliers.

Raw Data:

Salesperson	Region	Sales ($)	Customers
John	North	1000	50
Jane	South	NaN	40
Alice	East	1500	60
Bob	West	5000	200
Carol	North	200	5

Data Cleaning Steps:

1. **Treat Missing Values:**
   - Impute the missing Sales value for Jane using the mean sales value.

```python
mean_sales = df['Sales ($)'].mean()
df['Sales ($)'].fillna(mean_sales, inplace=True)
```

Treat Outliers:

- Identify outliers in Sales ($) using the IQR method.

```python
Q1 = df['Sales ($)'].quantile(0.25)
Q3 = df['Sales ($)'].quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
df = df[(df['Sales ($)'] >= lower_bound) & (df['Sales ($)'] <= upper_bound)]
```

Cleaned Data:

Salesperson	Region	Sales ($)	Customers
John	North	1000	50
Jane	South	1000	40
Alice	East	1500	60

Why Data Cleaning is Important

1. **Improves Data Quality**: Ensures the dataset is accurate and reliable.
2. **Enhances Model Performance**: Clean data leads to better model training and predictions.
3. **Reduces Bias**: Proper handling of missing values and outliers prevents skewed results.
4. **Saves Time**: Clean data reduces the need for repeated debugging and corrections during analysis.

DATA WRANGLING

Encoding Categorical Data

Categorical data represents discrete values that belong to specific categories. Since most machine learning algorithms work with numerical data, categorical data must be converted into numerical form.

Types of Categorical Data:

1. **Nominal Categorical Data:**
   - Categories with no inherent order or ranking.
   - Examples: Gender (Male, Female), Color (Red, Blue, Green), Country (USA, India, Japan).

2. **Ordinal Categorical Data:**
   - Categories with a meaningful order or ranking.
   - Examples: Education Level (High School, Bachelor's, Master's, PhD), Rating (Poor, Fair, Good, Excellent).

Encoding Techniques:

1. **Label Encoding:**
   - Assigns a unique integer to each category.
   - Suitable for ordinal data.
   - Example:
     - Education Level: Tall → 0, Medium → 1, Short → 2.

Height
Tall
Medium
Short

Height
0
1
2

2. **One-Hot Encoding:**
   - Creates binary columns for each category.
   - Suitable for nominal data.
   - Example:
     - Color: Red → [1, 0, 0], Blue → [0, 1, 0], Green → [0, 0, 1].

id	color
1	red
2	blue
3	green
4	blue

One Hot Encoding

id	color_red	color_blue	color_green
1	1	0	0
2	0	1	0
3	0	0	1
4	0	1	0

3. Ordinal Encoding:

- Assigns integers to categories based on their order.
- Suitable for ordinal data.
- Example:
  - Rating: Poor → 1, Good → 2, Very Good → 3, Excellent → 4.

Original Encoding	Ordinal Encoding
Poor	1
Good	2
Very Good	3
Excellent	4

4. Target Encoding (Mean Encoding):

- Replaces categories with the mean of the target variable for that category.
- Useful for high-cardinality categorical data.
- Example:
  - City: Replace each city name with the average house price in that city.

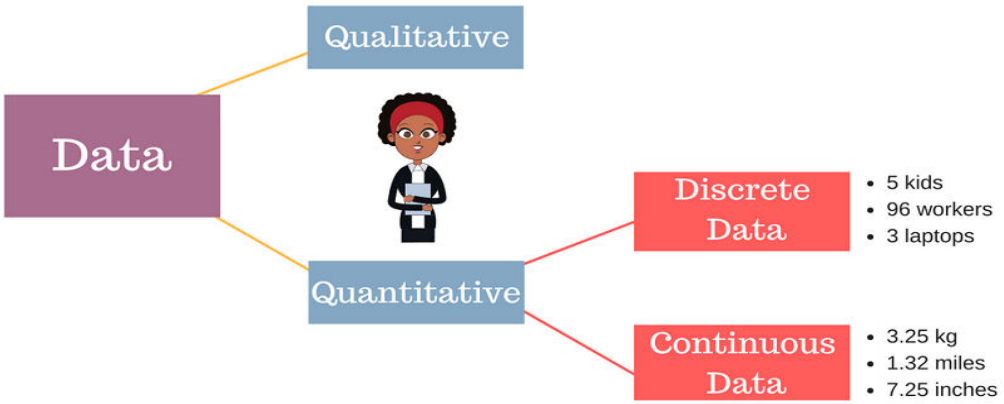
	Color	Target_2
0	Red	0
1	Red	0
2	Red	1
3	Red	0
4	Red	0
5	Green	0
6	Green	1
7	Green	0

	Color_Target_2
0	0.200000
1	0.200000
2	0.200000
3	0.200000
4	0.200000
5	0.333333
6	0.333333
7	0.333333

Data Types

1. Numerical Data:

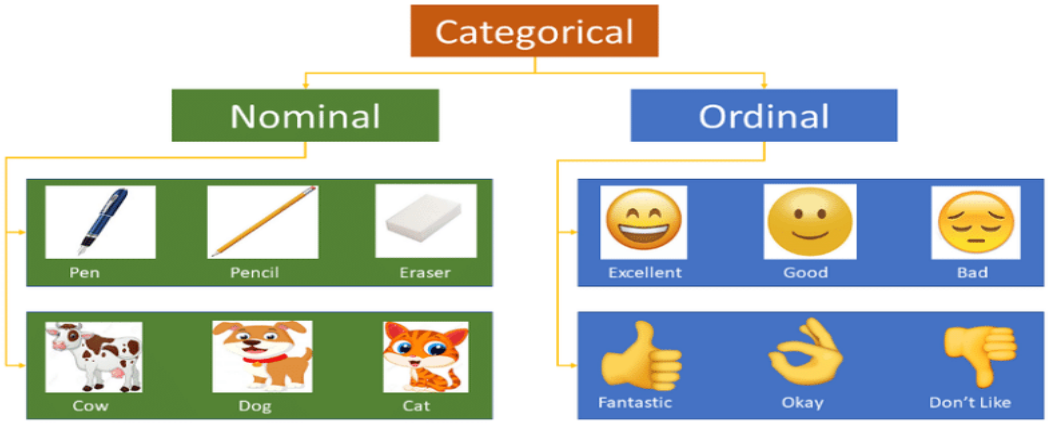
- **Represents quantifiable values.**
- **Continuous Data:** Can take any value within a range.
    - Examples: Height, Weight, Temperature.
- **Discrete Data:** Can only take specific, distinct values.
    - Examples: Number of children, Number of cars.



2. Categorical Data:

- **Represents discrete categories or groups.**

- **Nominal Categorical Data:** No inherent order.
    - Examples: Gender, Country, Color.
- **Ordinal Categorical Data:** Has a meaningful order.
    - Examples: Education Level, Rating, Income Level (Low, Medium, High).



Examples of Feature Engineering

1. **Feature Scaling:**
    - Scaling house sizes (in square feet) and prices (in dollars) to the same range for a regression model.
    - Standardizing age and income features for a clustering algorithm.
2. **Encoding Categorical Data:**
    - One-hot encoding "Country" (USA, India, Japan) for a classification model.
    - Label encoding "Education Level" (High School, Bachelor's, Master's, PhD) for an ordinal regression model.

Feature Engineering – Adding New Features and Modifying Data

Feature engineering is the process of **creating new features** or **modifying existing ones** to improve the performance of machine learning models. It involves using domain knowledge, creativity, and data manipulation techniques to extract more meaningful information from raw data.

Why is Feature Engineering Important?

1. **Improves Model Performance**: Well-engineered features can significantly enhance a model's ability to capture patterns and relationships in the data.
2. **Reduces Overfitting**: By creating relevant features, the model can focus on the most important aspects of the data, reducing the risk of overfitting.

3. **Handles Non-Linearity**: Many machine learning algorithms assume linear relationships. Feature engineering can introduce non-linear relationships that better represent the data.
4. **Domain-Specific Insights**: Incorporating domain knowledge into feature creation can lead to more interpretable and effective models.

Key Techniques in Feature Engineering

1. **Adding New Features**:
   - Create new features by combining or transforming existing ones.
   - Examples:
     - **Interaction Features**: Multiply or divide two features (e.g., Income / Family Size).
     - **Polynomial Features**: Create higher-order terms (e.g., x2x2, x3x3).
     - **Aggregation Features**: Summarize data (e.g., average, sum, max, min).
     - **Time-Based Features**: Extract day, month, or year from a date.
2. **Modifying Data**:
   - Transform or clean existing features to make them more useful.
   - Examples:
     - **Log Transformation**: Reduce skewness in data (e.g., log(Income).
     - **Binning**: Convert continuous data into discrete bins (e.g., age groups: 0-18, 19-35, 36-60, 60+).
     - **Handling Missing Values**: Impute missing data using mean, median, or predictive models.
     - **Normalization/Standardization**: Scale features to a common range.

Scenario: Predicting House Prices

Let's consider a scenario where we want to predict house prices based on features like size, location, number of bedrooms, and age of the house.

Raw Data:

Size (sq. ft.)	Location	Bedrooms	Age (years)	Price ($)
1500	Suburban	3	10	300,000
2000	Urban	4	5	500,000
1200	Rural	2	20	150,000

Feature Engineering Steps:

1. **Adding New Features**:
    - **Price per Square Foot**: Create a new feature by dividing Price by Size.

$$\text{Price per sq. ft.} = \frac{\text{Price}}{\text{Size}}$$

- **Location Encoding**: Convert the categorical feature Location into numerical values using one-hot encoding.
    - Suburban → [1, 0, 0]
    - Urban → [0, 1, 0]
    - Rural → [0, 0, 1]
- **Age Group**: Bin the Age feature into categories (e.g., 0-10, 11-20, 20+).

Modifying Data:

- **Log Transformation**: Apply a log transformation to Price to handle skewness.

$$\text{Log Price} = \log(\text{Price})$$

- **Normalization**: Normalize Size and Bedrooms to a range of [0, 1] to ensure equal contribution

Engineered Data:

Size (Normalized)	Location_Suburban	Location_Urban	Location_Rural	Bedrooms (Normalized)	Age Group	Price per sq. ft.	Log Price
0.3	1	0	0	0.5	0-10	200	12.61
0.6	0	1	0	1.0	0-10	250	13.12
0.1	0	0	1	0.0	20+	125	11.92

Why This is Important in the Scenario

1. **Price per Square Foot**: Captures the relationship between size and price, making it easier for the model to understand value.
2. **Location Encoding**: Converts categorical data into a numerical format that the model can process.
3. **Age Group**: Simplifies the Age feature into meaningful categories, reducing noise.
4. **Log Transformation**: Reduces skewness in the target variable (Price), improving model performance.
5. **Normalization**: Ensures that features like Size and Bedrooms contribute equally to the model.

By adding and modifying features, we provide the model with more meaningful and interpretable inputs, leading to better predictions and insights. This is the essence of feature engineering!

Feature Transformations

Feature transformation changes the representation of data while preserving its information.

(A) Log Transformation

Used to handle **skewed distributions** and reduce the impact of **outliers**.

◆ **Example:**

```
import numpy as np
import pandas as pd

df = pd.DataFrame({'salary': [2000, 5000, 10000, 50000, 100000]})
df['log_salary'] = np.log1p(df['salary'])  # log(1 + x) to avoid log(0)

print(df)
```

✔ Helps **normalize** skewed data.

(B) Square Root & Cube Root Transformation

Used when log transformation is too aggressive.

◆ **Example:**

```
df['sqrt_salary'] = np.sqrt(df['salary'])
df['cbrt_salary'] = np.cbrt(df['salary'])
```

✔ Useful for reducing large variations in data.

(C) Box-Cox & Yeo-Johnson Transformation

Used to make data **closer to normal distribution**.

◆ **Example:**

```
from scipy.stats import boxcox, yeojohnson

df['boxcox_salary'], _ = boxcox(df['salary'] + 1)  # Box-Cox needs positive values
df['yeojohnson_salary'], _ = yeojohnson(df['salary'])
```

✔ Box-Cox works only on **positive values**, Yeo-Johnson works on **both positive & negative** values.

(D) Binning (Discretization)

Converts continuous variables into discrete bins.

🔹 **Example:**

```
df['salary_category'] = pd.cut(df['salary'], bins=3, labels=['Low', 'Medium', 'High'])
```

✔ Useful for **decision trees** and **rule-based models**.

(E) Polynomial Features (Feature Expansion)

Used to **add complexity** by creating polynomial combinations.

🔹 **Example:**

```
from sklearn.preprocessing import PolynomialFeatures

poly = PolynomialFeatures(degree=2, include_bias=False)
X_poly = poly.fit_transform(df[['salary']])
```

✔ Helps in **non-linear models** like **polynomial regression**.

2. Feature Scaling

Feature scaling ensures all features are on the **same scale**, improving model training and convergence.

(A) Standardization (Z-Score Normalization)

Converts data to have **mean = 0** and **standard deviation = 1**.

🔹 **Formula:**

$$X_{\text{scaled}} = \frac{X - \text{mean}(X)}{\text{std}(X)}$$

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
df['scaled_salary'] = scaler.fit_transform(df[['salary']])
```

✔ Works well for **SVM, PCA, K-Means, Linear Regression**.

(B) Min-Max Scaling (Normalization)

Scales data between 0 and 1.

🔹 Formula:

$$X_{\text{scaled}} = \frac{X - X_{\min}}{X_{\max} - X_{\min}}$$

```
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()
df['normalized_salary'] = scaler.fit_transform(df[['salary']])
```

✔ Best for **Neural Networks and Gradient Descent-based models**.

(C) Robust Scaling (Resistant to Outliers)

Scales data using **median and interquartile range (IQR)**.

🔹 **Formula:**

$$X_{\text{scaled}} = \frac{X - \text{median}(X)}{\text{IQR}(X)}$$

```
from sklearn.preprocessing import RobustScaler

scaler = RobustScaler()
df['robust_scaled_salary'] = scaler.fit_transform(df[['salary']])
```

✔ Works well when **data has outliers**.

Conclusion

✅ **Feature Transformation** helps make data more suitable for learning by changing its representation.
✅ **Feature Scaling** ensures models perform efficiently by making data comparable.
✅ **Choice of transformation/scaling depends on model requirements and data distribution.**

Feature Selection

Feature selection is the process of selecting the most relevant features (variables) from a dataset to improve model performance, reduce overfitting, and enhance interpretability. It helps in removing irrelevant, redundant, or noisy features, making the model more efficient and effective.

Types of Feature Selection Methods

1. Filter Methods

Filter methods evaluate the relevance of features based on statistical measures and select features independently of the machine learning algorithm. They are computationally efficient and work well for high-dimensional datasets.

Common Techniques:

1. **Correlation Coefficient**:
   - Measures the linear relationship between features and the target variable.
   - Example: Pearson correlation for continuous data, Spearman correlation for ordinal data.
   - Features with high correlation (absolute value) are selected.

2. **Chi-Square Test**:
   - Used for categorical features to test the independence between the feature and the target.
   - Example: Selecting categorical features for classification tasks.
3. **Mutual Information**:
   - Measures the dependency between features and the target variable.
   - Suitable for both continuous and categorical data.
4. **VarianceThreshold**:

```python
from sklearn.feature_selection import VarianceThreshold
selector = VarianceThreshold(threshold=0.1)
X_selected = selector.fit_transform(X)
```

   - Removes features with low variance (e.g., constant or near-constant features).

Advantages:

- Fast and scalable.
- Independent of the machine learning algorithm.

Disadvantages:

- Ignores feature interactions.
- May not capture complex relationships.

2. Wrapper Methods

Wrapper methods evaluate subsets of features by training and testing a specific machine learning model. They use the model's performance as the evaluation criterion and search for the optimal feature subset.

Common Techniques:

1. **Forward Selection**:
   - Starts with no features and iteratively adds the best-performing feature.

```python
from sklearn.feature_selection import SequentialFeatureSelector
sfs = SequentialFeatureSelector(estimator, n_features_to_select=5, direction='forward')
sfs.fit(X, y)
```

Backward Elimination:

- Starts with all features and iteratively removes the least significant feature.

```python
sfs = SequentialFeatureSelector(estimator, n_features_to_select=5, direction='backward')
sfs.fit(X, y)
```

Recursive Feature Elimination (RFE):

- Recursively removes the least important features based on model weights (e.g., coefficients in linear models).

```python
from sklearn.feature_selection import RFE
rfe = RFE(estimator, n_features_to_select=5)
rfe.fit(X, y)
```

1. **Exhaustive Search**:
    - Evaluates all possible feature subsets (computationally expensive).

Advantages:

- Considers feature interactions.
- Optimizes model performance.

Disadvantages:

- Computationally expensive.
- Prone to overfitting.

Embedded Methods

Embedded methods perform feature selection as part of the model training process. They combine the advantages of filter and wrapper methods by considering feature interactions and being computationally efficient.

Common Techniques:

1. **Lasso Regression (L1 Regularization)**:

- Adds a penalty term to the loss function, forcing some feature coefficients to zero.

```python
from sklearn.linear_model import Lasso
lasso = Lasso(alpha=0.01)
lasso.fit(X, y)
selected_features = lasso.coef_ != 0
```

Ridge Regression (L2 Regularization):

- Adds a penalty term to shrink coefficients but does not force them to zero.

```python
from sklearn.linear_model import Ridge
ridge = Ridge(alpha=0.01)
ridge.fit(X, y)
```

Tree-Based Methods:

- Use feature importance scores from tree-based models (e.g., Random Forest, Gradient Boosting).

```python
from sklearn.ensemble import RandomForestClassifier
model = RandomForestClassifier()
model.fit(X, y)
importances = model.feature_importances_
```

1. **Elastic Net**:
   - Combines L1 and L2 regularization for feature selection.

Advantages:

- Computationally efficient.
- Considers feature interactions.

Disadvantages:

- Model-specific (depends on the algorithm used).

Comparison of Feature Selection Methods

Method	Advantages	Disadvantages	Use Case
Filter Methods	Fast, scalable, independent of model	Ignores feature interactions	High-dimensional data, preprocessing
Wrapper Methods	Considers feature interactions, optimizes performance	Computationally expensive, prone to overfitting	Small datasets, model-specific tuning
Embedded Methods	Efficient, considers feature interactions	Model-specific	Regularized models, tree-based models

Scenario: Feature Selection for a Classification Problem

Consider a dataset with 20 features for predicting whether a customer will churn.

Steps:

1. **Filter Method**:
    - Use correlation to select the top 10 features most correlated with the target variable.
2. **Wrapper Method**:
    - Use Recursive Feature Elimination (RFE) with a logistic regression model to select the top 5 features.
3. **Embedded Method**:
    - Use Lasso regression to identify features with non-zero coefficients.

Result:

- Filter method reduces features to 10.
- Wrapper method further reduces features to 5.
- Embedded method confirms the importance of the selected features.

Why Feature Selection is Important

1. **Improves Model Performance**: Reduces overfitting and enhances generalization.

2. **Reduces Training Time**: Fewer features mean faster model training.
3. **Enhances Interpretability**: Simplifies the model and makes it easier to understand.
4. **Handles High-Dimensional Data**: Essential for datasets with many features (e.g., text, images).

Dimensionality Reduction Techniques – PCA

Dimensionality reduction is the process of reducing the number of features (variables) in a dataset while retaining as much meaningful information as possible. It is crucial for improving model performance, reducing computation time, and removing noise from data.

1. Principal Component Analysis (PCA)

PCA is a statistical technique used to transform high-dimensional data into a lower-dimensional space while preserving the variance of the original dataset. It finds a new set of axes (principal components) that maximize variance and project the data onto these axes.

📌 **Steps in PCA**

Step 1: Standardize the Data

Since PCA is affected by different scales of features, the data should be standardized to have zero mean and unit variance. This ensures that all features contribute equally.

Formula for standardization:

$$X' = \frac{X - \mu}{\sigma}$$

where X is the original feature, μ is the mean, and σ is the standard deviation.

🔷 **Python Code Example**

```python
from sklearn.preprocessing import StandardScaler
import numpy as np

X = np.array([[2.5, 3.1], [1.2, 0.8], [4.5, 5.2], [3.2, 3.8]])
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

Step 2: Compute the Covariance Matrix

The covariance matrix represents how features vary together. It is calculated as:

$$C = \frac{1}{n-1} X^T X$$

where X is the standardized data matrix.

🔷 **Python Code Example**

```python
cov_matrix = np.cov(X_scaled.T)
print(cov_matrix)
```

Step 3: Compute Eigenvalues and Eigenvectors

Eigenvalues measure the variance explained by each principal component, while eigenvectors define the direction of these components.

◆ **Python Code Example**

```python
eigenvalues, eigenvectors = np.linalg.eig(cov_matrix)
print("Eigenvalues:", eigenvalues)
print("Eigenvectors:\n", eigenvectors)
```

Step 4: Select Top k Principal Components

Sort the eigenvalues in descending order and select the top k components that explain the most variance. The explained variance ratio is given by:

$$\text{Explained Variance} = \frac{\lambda_i}{\sum \lambda}$$

where λi are the eigenvalues.

◆ **Python Code Example**

```python
explained_variance_ratio = eigenvalues / np.sum(eigenvalues)
print("Explained Variance Ratio:", explained_variance_ratio)
```

Step 5: Project Data onto the New Feature Space

The original data is transformed using the selected eigenvectors to obtain reduced dimensions.

🔷 Python Code Example

```
# Project onto first two principal components
X_pca = np.dot(X_scaled, eigenvectors[:, :2])
print(X_pca)
```

📌 Variants of PCA

Sparse PCA

- Introduces sparsity constraints on principal components to select fewer influential features.
- Useful when dealing with highly correlated features.
  🔷 **Use Case**: Gene expression data, where only a few genes contribute to a condition.

```python
from sklearn.decomposition import SparsePCA
sparse_pca = SparsePCA(n_components=2)
X_sparse_pca = sparse_pca.fit_transform(X_scaled)
```

Kernel PCA

- Extends PCA to non-linear data using kernel functions (e.g., polynomial, RBF, sigmoid).
  🔷 **Use Case**: Facial recognition, where data is non-linearly distributed.

```python
from sklearn.decomposition import KernelPCA
kpca = KernelPCA(n_components=2, kernel='rbf')
X_kpca = kpca.fit_transform(X_scaled)
```

Singular Value Decomposition (SVD)

SVD is another dimensionality reduction technique that decomposes a matrix AAA into three matrices:

$$A = U\Sigma V^T$$

where:

- U = Left singular vectors (orthogonal matrix)
- Σ = Diagonal matrix of singular values
- V^T = Right singular vectors (orthogonal matrix)

📌 **Steps in SVD**

Step 1: Compute SVD

🔷 **Python Code Example**

```python
from numpy.linalg import svd

U, S, VT = svd(X_scaled)
print("U:\n", U)
print("Sigma:\n", S)
print("V^T:\n", VT)
```

Step 2: Select k Largest Singular Values

Singular values in Σ represent variance in data. We select the top k values to reduce dimensionality.

```python
k = 2
X_svd = np.dot(U[:, :k], np.diag(S[:k]))
print(X_svd)
```

📌 **Differences Between PCA and SVD**

Feature	PCA	SVD
Input Data	Covariance Matrix	Original Matrix
Purpose	Reduces dimensions while maximizing variance	Decomposes matrix for various applications
Computation	Uses eigenvalues & eigenvectors	Uses singular values & orthogonal matrices
Suitability	Used for feature reduction	Used for feature extraction & noise reduction

📌 **Key Takeaways**

✅ **PCA is a variance-based method that projects data onto principal components.**
✅ **Sparse PCA adds sparsity for better feature selection.**
✅ **Kernel PCA is useful for non-linear transformations.**
✅ **SVD decomposes a matrix into meaningful components and is useful for data compression.**

Non-Negative Matrix Factorization (NMF)

📌 What is NMF?

Non-Negative Matrix Factorization (NMF) is a dimensionality reduction technique used to decompose a non-negative matrix into two lower-dimensional non-negative matrices. Unlike PCA or SVD, NMF ensures that all elements remain non-negative, making it useful for interpretability in applications like topic modeling, image processing, and recommender systems.

📌 Mathematical Representation

Given a non-negative matrix A of size m×n, NMF factorizes it into two non-negative matrices:

$$A \approx W \times H$$

where:

- W is an m×k matrix (basis matrix).
- H is a k×n matrix (coefficient matrix).
- k is the number of latent features (usually k<n for dimensionality reduction).

📌 Steps in NMF

- **Step 1: Prepare the Data**
- Ensure that all elements in the matrix are non-negative. NMF does not work with negative values.

```python
import numpy as np
from sklearn.decomposition import NMF

# Example data (must be non-negative)
A = np.array([[2, 3, 4],
              [5, 6, 7],
              [8, 9, 10]])

# Number of components (latent features)
k = 2
```

Step 2: Apply NMF

```python
nmf = NMF(n_components=k, random_state=42)
W = nmf.fit_transform(A)
H = nmf.components_

print("W Matrix:\n", W)
print("H Matrix:\n", H)
```

Step 3: Reconstruct the Original Matrix

To approximate A, multiply W and H.

```python
A_approx = np.dot(W, H)
print("Reconstructed A:\n", A_approx)
```

The reconstructed matrix A_approx will be close to A, but with reduced dimensions.

📌 **Applications of NMF**

✅ **Topic Modeling** – Extracts topics from text documents.
✅ **Recommender Systems** – Used for collaborative filtering (e.g., movie recommendations).
✅ **Image Processing** – Feature extraction in images.
✅ **Bioinformatics** – Identifying gene expression patterns.

Interview Question

Creating Training and Test Sets

Q1. Why do we split data into training and test sets?
A1. To evaluate model performance on unseen data and prevent overfitting.

Q2. What is the typical ratio for splitting data into training and test sets?
A2. Common splits:

- **80/20** (80% training, 20% testing)
- **70/30** (for larger datasets)
- **90/10** (when data is limited)

Q3. What is stratified sampling, and when should you use it?
A3. Stratified sampling ensures that the train-test split maintains the same class distribution as the original dataset. It is useful for **imbalanced classification problems**.

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, stratify=y, random_state=42)
```

Data Scaling and Normalization

Q4. What is the difference between Standardization and Normalization?
A4.

- **Standardization (Z-score scaling):** Converts data to have mean = 0 and standard deviation = 1.
- **Normalization (Min-Max scaling):** Rescales data to a fixed range, typically [0,1].

Q5. When should you use Standardization vs. Normalization?
A5.

- **Standardization** → When data follows a normal distribution. Used in linear models and SVM.
- **Normalization** → When data is skewed or has different ranges. Used in deep learning and KNN.

Q6. How do you standardize and normalize data in Scikit-Learn?

```python
from sklearn.preprocessing import StandardScaler, MinMaxScaler

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

normalizer = MinMaxScaler()
X_normalized = normalizer.fit_transform(X)
```

Feature Engineering

Q7. What is feature engineering, and why is it important?
A7. Feature engineering involves creating new features or modifying existing ones to improve model performance. It is crucial because better features lead to better predictions.

Q8. Give examples of feature engineering techniques.
A8.

- **Adding new features** (e.g., creating "age groups" from raw age values).
- **Combining features** (e.g., BMI = weight / height²).
- **Extracting datetime features** (e.g., day of the week from a timestamp).
- **Log transformation** (for skewed data).

Data Cleaning (Handling Missing Values and Outliers)

Q9. How do you handle missing values in a dataset?
A9.

- **Drop missing values** (if the missing rate is very high).
- **Impute with mean/median/mode** (for numerical data).
- **Use forward/backward fill** (for time-series data).
- **Use a predictive model** (e.g., KNN imputation).

```
from sklearn.impute import SimpleImputer
imputer = SimpleImputer(strategy='mean')
X_imputed = imputer.fit_transform(X)
```

Q10. How do you detect and handle outliers?
A10.

- **Z-score method** (removing values beyond ±3 standard deviations).
- **IQR method** (removing values outside [Q1 - 1.5*IQR, Q3 + 1.5*IQR]).
- **Winsorization** (capping extreme values).

```
import numpy as np
Q1 = np.percentile(X, 25)
Q3 = np.percentile(X, 75)
IQR = Q3 - Q1
X_filtered = X[(X > Q1 - 1.5*IQR) & (X < Q3 + 1.5*IQR)]
```

Data Wrangling (Encoding, Feature Transformations, Scaling)

Q11. What are different encoding techniques for categorical data?
A11.

- **One-Hot Encoding** (for nominal data)
- **Label Encoding** (for ordinal data)
- **Target Encoding** (when the categorical variable has many levels)

```
from sklearn.preprocessing import OneHotEncoder, LabelEncoder

ohe = OneHotEncoder()
X_ohe = ohe.fit_transform(X)

le = LabelEncoder()
X_le = le.fit_transform(X)
```

Q12. What are common feature transformation techniques?
A12.

- **Log transformation** (for skewed data).
- **Power transformations (Box-Cox, Yeo-Johnson)**.
- **Polynomial feature expansion** (for linear models)

```
from sklearn.preprocessing import PowerTransformer
pt = PowerTransformer()
X_transformed = pt.fit_transform(X)
```

Feature Selection

Q13. What are the three main types of feature selection methods?
A13.

1. **Filter Methods** (based on statistical tests)
    - Pearson Correlation
    - Chi-Square test
    - Mutual Information
2. **Wrapper Methods** (based on model performance)
    - Recursive Feature Elimination (RFE)
    - Forward/Backward Selection

3. **Embedded Methods** (built into model algorithms)
    - Lasso Regression (L1 regularization)
    - Decision Trees (Feature Importance)

Q14. How do you perform feature selection in Scikit-Learn?

```
from sklearn.feature_selection import SelectKBest, chi2

selector = SelectKBest(score_func=chi2, k=10)
X_selected = selector.fit_transform(X, y)
```

Dimensionality Reduction

Principal Component Analysis (PCA)

Q15. What is PCA, and how does it work?
A15. PCA reduces dimensionality by projecting data onto principal components that maximize variance. It helps remove noise and speed up training.

Q16. What is the difference between Sparse PCA and Kernel PCA?
A16.

- **Sparse PCA**: Uses L1 regularization to force some principal components to be zero.
- **Kernel PCA**: Uses kernel functions (e.g., RBF) to perform PCA in a higher-dimensional space.

```
from sklearn.decomposition import PCA, KernelPCA
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)

kpca = KernelPCA(n_components=2, kernel='rbf')
X_kpca = kpca.fit_transform(X)
```

Singular Value Decomposition (SVD)

Q17. What is Singular Value Decomposition (SVD), and where is it used?
A17. SVD factorizes a matrix into three matrices: $A=U\Sigma V^T$. It is widely used in **recommendation systems and latent factor models**.

```
from sklearn.decomposition import TruncatedSVD
svd = TruncatedSVD(n_components=5)
X_svd = svd.fit_transform(X)
```

Non-Negative Matrix Factorization (NMF)

Q18. What is Non-Negative Matrix Factorization (NMF), and how is it different from PCA?
A18.

- NMF factorizes a matrix into non-negative components (unlike PCA, which allows negative values).
- Used in **topic modeling, image processing, and document clustering**.

```
from sklearn.decomposition import NMF
nmf = NMF(n_components=5)
X_nmf = nmf.fit_transform(X)
```

Blogs:

1. **Data Preprocessing and Feature Engineering in Machine Learning**
   https://magnimindacademy.com/blog/data-preprocessing-and-feature-engineering-in-machine-learning/

2. **Dimensionality Reduction in Machine Learning and Data Science**
   https://saeedmirshekari.com/blog/dimensionality-reduction-in-machine-learning-and-data-science/

3. **Introduction to Dimensionality Reduction**
   https://www.geeksforgeeks.org/dimensionality-reduction/.

4. **Feature Selection and Dimensionality Reduction**
   https://leehanchung.github.io/blogs/2021/04/12/Feature-Selection/

5. **The Art of Feature Engineering in Data Preprocessing**
   https://mlres.net/the-art-of-feature-engineering-in-data-preprocessing/

Kaggle Notebooks:

1. **Preprocessing Images with Dimensionality Reduction**
   https://www.kaggle.com/code/hamishdickson/preprocessing-images-with-dimensionality-reduction

2. **Dimensionality Reduction Techniques**
   https://www.kaggle.com/code/karltonkxb/dimensionality-reduction-techniques

3. **Data Preprocessing and Feature Engineering**
   https://www.kaggle.com/code/marytong/data-preprocessing-feature-engineering

4. **Housing Price Prediction: Advanced Regression Techniques**
   https://github.com/ShovalBenjer/Housing_Price_Prediction_Advanced_Regresson_Kaggle